Aufgabe 1 - den Code in Komponenten aufteilen

In dieser Aufgabe teilst du den Code der App.js in mehrere Dateien auf, wobei einzelne Teile als neue "Komponenten" gespeichert werden. Komponenten bündelnd UI (HTML) und Funktionalität (JS) in einer gemeinsamen Funktion und bilden die Grundbausteine der App. Dadurch wird der Code zunächst einmal übersichtlicher.

1.1 Footer-Komponente erstellen

1. Lege eine neue Datei "Footer.jsx" (Grossschreibung) im Ordner src an. In diese schreibst du eine Funktion mit dem Namen "Footer". Orientiere dich an der App.js.

```
function App() {
  return ()
}

export default App;
```

2. Kopiere aus der App.js den Code vom Footer und füge ihn im return-Statement deiner Footer-Komponente und zwischen die runden Klammern ein. Belasse die Zeilen mit Kommentaren bezüglich der Footer-Komponente in der App.js.

3. Importiere nun deine neue Footer-Komponente in App.js, indem du diese Zeile ganz oben bei den Imports einfügst:

```
import Footer from "./src/Footer":
```

4. Füge nun zwischen die beiden Kommentarzeilen eine neue Zeile ein. Diese soll nicht Teil eines Kommentars sein. In diese Zeile schreibst du

```
<Footer />
```

Damit referenzierst du deine neue Komponente. Du kannst dir das als Variable vorstellen. Die `<Footer/>` - Komponente ersetzt also den alten Code an dieser Stelle, welchen du ausgelagert hast. Beachte, dass Komponenten im Vergleich zu HTML-Elementen immer gross geschrieben werden - damit soll klar werden, was ein HTML-Element (von HTML) und was eine Komponente (von React / JSX) ist. Danach kannst du die Kommentarzeilen entfernen.

5. Test: Wird der Footer im Browser angezeigt? Falls es Fehler gibt, liegt es vielleicht am fehlenden Import oder der Schreibweise?

1.2 Tabellen-Komponente erstellen

1. Mache nun das Gleiche für die Tabelle. Das äussere Element soll das div mit `className="TableWrapper"` sein. Du kannst die Komponente frei benennen, achte aber darauf den selben Namen für den Import und die Referenzierung zu verwenden.

2. Teste das Ergebnis. Wird die Tabelle wieder angezeigt? Falls es Fehler gibt, hast du eventuell einen Teil vergessen zu kopieren und manche Elemente sind nicht geeschlossen?

1.3 - Dynamische Tabelle mit JSON-Datei

Aktuell ist die Tabelle statisch. Für gewöhnlichen kommen Daten aber als JSON-Datei vom Server. Daher muss die UI dynamisch auf die Datenmenge reagieren. Zudem soll die Tabelle später auch dynamisch gefiltert werden können. Dazu braucht es keine Server-Abfrage, aber React State.

1. Importiere die JSON Datei mit den Daten, indem du die Zeile `import data from "./data/tableData";` ganz an den Anfang der Datei und ausserhalb der Funktion / Komponente schreibst. Orientiere dich ggf. an der App.js.

2. Finde in deiner Tabellen-Komponente das tbody-Element (`<tbody id="TableBody">`) und lösche den gesammten Code zwischen diesem Start- und dem Endtag (`/body`).

3. Setze nun diesen Code zwischen die Tags. Achte auf die geschweiften Klammern. Dieser erzeugt die Tabelle mithilfe von map():

```
{data.map((row, index) => (
<tr key={index}>
    <td>{row.Kanton}</td>
    <td>{row.Titel}</td>
</tr>
))}
```

Viel weniger Code!

4. Die Tabelle ist noch unvollständig. Was fehlt noch? Schau ggf. in die JSON-Datei.

Aufgabe 2 - Props für State-Variablen

In dieser Aufgabe leitest du Daten mit sogn. "Props" von einer Komponente zu einer anderen Komponente. Bei diesen Daten handelt es sich ausschliesslich um State-Variablen und setState-Funktionen - d.h. Daten, welche sich bei der Benutzung der App ändern können. Statische Daten, z.B. Variablen, werden hingegen durch import / export geteilt. Props dienen also dazu, dass eine Interaktion oder Datenmanipulation auch in anderen Teilen (Komponenten) der App ankommt und die UI auf diese Änderung reagieren kann.

2.1 - Überblick verschaffen

Verschaff dir nochmal einen kurzen Überblick über den aktuellen Stand der App. Die Interaktionselemente und React State sind in der App.js. Die Tabelle ist in einer eigenen Komponente. Der Footer mit Button ist ebenfalls ausgelagert. Aktuell reagiert die Tabelle noch nicht auf die Änderung von State. Es fehlt zum

einen, eine Filterfunktion, welche die Tabelle durch Auswahl oder Eingabe in die Interaktionselemente filtert. Es fehlen aber auch React Props, um die State-Variablen an die Tabellen-Komponente leiten, wo sie für die Filterung genutzt werden sollen.

2.2 - Filterfunktion einfügen

Kopiere diesen Code in die Tabellen-Komponente, vor dem return-Statement.

```
const filteredData = data.filter((item) => {
  const filteredBySuchbegriff =
    !suchbegriff ||
    item.Beschreibung.toLowerCase().includes(suchbegriff.toLowerCase());
  const filteredByKanton = !kantonAuswahl || item.Kanton === kantonAuswahl;
  const filteredByServicetype =
    !serviceTypAuswahl || item.ServiceTyp === serviceTypAuswahl;

  return filteredBySuchbegriff && filteredByKanton && filteredByServicetype;
});
```

Dies ist die Filterfunktion für die Tabelle. Data kommt aus der JSON-Datei, "suchbegriff", "kantonAuswahl", "serviceTypAuswahl" referenzieren die jeweiligen State-Variablen aus den Hooks in der App.js. Du bekommst eine Fehlermeldung, weil diese Variablen (noch) nicht in der Tabellenkomponente deklariert sind. Aktuell ist die Komponente auch noch nicht über Props mit der App.js verbunden.

2.3 - Props definieren

Finde die erste Zeile deiner Tabellen-Komponente (`function deinKomponentenName (){...`). In die runden Klammern schreibe `{suchbegriff, kantonAuswahl, serviceTypAuswahl}`. Damit sagst du, dass diese drei Variablen über Props in die Komponente / Funktion kommen. Speichere die Datei.

In der App.js musst du nun die drei State-Variablen auch an die Komponente übergeben. Finde die Zeile im return()-Statement der App.js, wo die Tabellenkomponente referenziert ist. Deklariere dort die Props wie Attribute.

```
// Beispiel

<MeineTabellenkomponente suchbegriff={suchbegriff} usw. >
```

Teste danach die Filter, diese sollten nun die Tabelle filtern. Beachte, dass du Props immer auf zwei "Seiten" definieren musst - dort wo sie an die Komponente gegeben werden (hier in der App.js) und dort wo sie in der Komponente gelesen werden (in der Tabellen-Komponente). Wenn du eine Seite vergisst, kommen die Daten nicht an.

Aufgabe 3 - Props für setState-Funktionen

In dieser Aufgabe bindest du auch die Footer-Komponente an. Egal ob du State-Varaiblen, setState-Funktionen oder beides mit Komponenten teils, Props funktionieren immer gleich. Teile zwischen Komponenten nur das, was wirklich gebraucht wird und nicht einfach *alle* Daten.

3.1 - Überblick verschaffen

Der Button im Footer soll die Filter zurücksetzten. Für jedes Auswahlmenü ist eine "Alle"-Option definiert, welche einen leeren String als Wert hat (siehe Value-Attribut). Auch ein leeres Eingabefeld hat diesen Wert. Die Filterfunktionen sind so geschrieben, dass ein leerer String dazu führt, dass *alle* Daten der Kategorie angezeigt werden, also kein Filter zur Anwendung kommt. Der Button soll auf den initialen State (Leere Strings für alle Filter) zurücksetzen.

3.2 - Props definieren

Gib die drei setState-Funktionen aus der App.js an die Footer-Komponente. Beachte dabei, dass du Props wieder auf beiden "Seiten" definierst - in der App.js (wie Attribute) und in der Footer-Komponente (wie Parameter). Vergleiche ggf. wie du das in Aufgabe 2.3 gemacht hast.
Statt Variablen teilst du jetzt die jweiligen setState-Funktionen - oder besser gesagt: eine Referenz zu diesen. Damit kann der Footer die Funktionen aufrufen. Das kommt im nächsten Schritt.

3.3 - Event Handler

Nun fehlt nur noch der EventHandler. Verwende im Button "onClick" und rufe dort alle drei Funktionen mit einem leeren String als Argument auf. Wenn du eine Pfeilfunktion definierst, beachte die {}-Klammer für mehrzeiligen Code:

```
<button onClick={() => {
    setState-Funktion 1;
    setState-Funktion 2;
    setState-Funktion 3;
}}>
```

Teste, ob das Zurücksetzen der Filter mit dem Button funktioniert und die Tabelle reagiert.

Aufgabe 4 - Header auslagern

Auch der Header (das `<header>`-Element und alle untergeordneten Elemente) kann in eine eigene Komponente ausgelagert werden. Erstelle eine neue Datei für die Komponente und passe dann die App.js sowie die neue Komponente an. Gib die benötigten Daten via Props an den Header. Die Hooks belässt du in der App.js. Teste danach, ob die Filter und die Tabelle weiter funktionieren.

Achtung: Du musst auch die Import-Statements für "logo" und "data" sowie die Zeile zur Generierung der Kantonsliste (`const kantone = [...new Set...`) in die neue Komponente verschieben.